

Editing Your Website

A plain-English guide for gudulopes.com — using VS Code + GitHub Desktop

Updated 24 Jul 2026 — YouTube embeds now use the privacy-friendly youtube-nocookie.com domain, and the homepage video is set.

The one big idea, first. Your website's real files live on GitHub (a repository called **gudulopes-site**). The website gudulopes.com simply shows whatever is in GitHub. You keep a copy of those files in a folder on your PC, edit them in Visual Studio Code, and use GitHub Desktop to send your changes back up. About a minute after you send them, the live site updates by itself.

Your three tools and what each one does

Tool	Its job
The folder on your PC	Your working copy of the site. This is what you edit.
Visual Studio Code	The editor — where you actually change text, videos, etc.
GitHub Desktop	The bridge — pulls the latest down, pushes your changes up to make them live.

You cannot break anything permanently. GitHub keeps a full history of every change, and VS Code has unlimited undo (Ctrl+Z). If something goes wrong, the old version is always recoverable.

Part 1 — One-time setup (do this once)

1. In GitHub Desktop, make sure the current repository is **gudulopes-site**. (Top-left “Current repository”. If it's not there yet: File → Clone repository → pick GLS9000/gudulopes-site.)
2. Click “Open in Visual Studio Code” on the GitHub Desktop screen. VS Code opens your whole site folder.
3. Recommended: in VS Code, install the “Live Preview” extension (squares icon on the left, search Live Preview, Install). It lets you see your page update as you type.

Not sure which folder is the real one? In GitHub Desktop use Repository → Show in Explorer to open the exact synced folder. Always edit that one — not an old download.

Part 2 — Your everyday editing loop

Every change follows these five steps. Learn this once; the rest of the guide is just “which file, which spot.”

1. **Pull first.** Open GitHub Desktop, click Fetch origin, then Pull if it offers.
2. **Edit in VS Code.** Open the file, make your change, save with Ctrl+S. (Saving only changes your PC.)
3. **Check.** In GitHub Desktop, the Changes panel lists what you touched — old text in red, new in green.
4. **Commit.** Type a short Summary (e.g. “Add Sleep Tight video + story”) and click Commit to main.
5. **Push.** Click Push origin. Wait ~1 minute, then refresh gudulopes.com — your change is live.

Golden rule: only edit on your PC — never on github.com too. Editing in both places makes the copies disagree (“merge conflict”). One place only, and always Pull before you start.

Two VS Code tricks. Ctrl+F searches the file you're in; Ctrl+Shift+F searches every file at once (great for finding all VIDEO_ID spots). Right-click index.html → Show Preview / Open with Live Preview to watch the real styled page update as you type.

What are those `<!-- EDIT: ... -->` lines? Friendly signposts left in the files just for you — they mark where it's safe to change things. Anything between `<!--` and `-->` is a private note; it never appears on the website.

Part 3 — Add or change a VIDEO

Which file?

The video you want to change	File to open in VS Code
A song on Door of Sleep	door-of-sleep.html
A song on Door of Forgetting	door-of-forgetting.html
A song on Door of Madness	door-of-madness.html
A song on Door of Death	door-of-death.html
The big “Watch” video on the homepage	index.html

What is a “video ID”?

Every YouTube video has a short code in its web address. When you open a video on YouTube, the address looks like this:

```
https://www.youtube.com/watch?v=aUew0fVNL-Y&other-stuff
```

The ID is the part right after **v=** — here, **aUew0fVNL-Y**. Ignore the & and anything after it.

Where to put it

In VS Code, open the right file and press Ctrl+F, type **VIDEO_ID**, Enter. You'll land on a line like “Before” below. Replace the words VIDEO_ID with your real code. Change nothing else, then save.

*Note the domain is **youtube-nocookie.com** (the privacy-friendly version — no tracking cookies until a visitor clicks play). This is normal; you still only swap the ID.*

BEFORE

```
<iframe src="https://www.youtube-nocookie.com/embed/VIDEO_ID" title="Sleep Tight" ...>
```

AFTER (REAL ID PASTED IN)

```
<iframe src="https://www.youtube-nocookie.com/embed/aUew0fVNL-Y" title="Sleep Tight" ...>
```

Keep the quotation marks and the rest of the line. Only the code between **/embed/** and the closing **"** gets replaced.

Paste just the ID — don't paste YouTube's full “Embed” code

Always use the method above: paste **only the ID**. The line is already set up to load fast, stay responsive, and protect visitor privacy.

Avoid pasting YouTube's whole “Embed” snippet over the `<iframe>` line. It would drag back the tracking youtube.com domain (undoing the privacy setting) and its fixed

560×315 size (which breaks the responsive sizing on phones).

The homepage “Watch” video is already set (to aUew0fVNL-Y). So searching VIDEO_ID in index.html now lands only on the explanatory comment, not an empty embed — to change the homepage video, just replace that ID on the “Watch” iframe. The remaining VIDEO_ID placeholders are on the four door pages.

Until you paste a real ID on a door page, that video shows “Video unavailable” — normal, and only you see it in preview.

Part 4 — Add or change TEXT

Text lives in the page it appears on: homepage tagline/bio → index.html; a door's intro → that door's file. Search for words you can see on the site, or the nearest EDIT signpost, then type between the > and the </.

BEFORE

```
<!-- EDIT: TAGLINE -->
<p class="tagline">Composer & producer from Ghent...</p>
```

AFTER

```
<!-- EDIT: TAGLINE -->
<p class="tagline">New album "Door of Sleep" - out now.</p>
```

Don't touch the tags. The bits in angle brackets like <p class="tagline"> and </p> are the container. Type only between them.

Writing a song's STORY

On a door page, each song has an empty story box below its video. Search the song title, then:

```
BEFORE (EMPTY — NOTHING SHOWS YET)
<div class="track-story"></div>

AFTER — TYPE INSIDE, WRAPPED IN <p>...</p>
<div class="track-story"><p>This track began as a lullaby...</p></div>
```

Two paragraphs? Use two <p>...</p> blocks in a row.

Part 5 — Add or replace a COVER IMAGE

Because you work locally, images are the easiest thing of all: just put the file in the folder. No upload dialog.

1. Shrink the image first (see below). Name it exactly as the table below.
2. Drop it into the assets/images folder — drag it into VS Code's file list on the left, or open that folder in Windows Explorer and paste it in.
3. Commit + Push in GitHub Desktop. The cover appears automatically (the page already looks for that file name).

Page	Name the square image exactly...
Door of Sleep	sleep-cover.jpg (done)
Door of Forgetting	forgetting-cover.jpg
Door of Madness	madness-cover.jpg

Door of Death	death-cover.jpg
Boku no Sekai	kompas-cover.jpg (done)

Shrink images first! Photos from a phone or AI tools are often 3-5 MB — too heavy, and they slow the site. Aim for a square image ~1000×1000 px, under ~300 KB. Any free online “image compressor” does it in seconds. (Claude can also shrink them if you hand over the original.)

Part 6 — Change a LINK (Spotify, Instagram, Apple Music...)

Streaming links are near the top of index.html (search EDIT: STREAMING LINKS); social links are in every page's footer (EDIT: SOCIAL LINKS). Change only the address inside the quotes after href=.

```
BEFORE
<a class="stream-link" href="https://open.spotify.com/artist/OLD" ...>

AFTER
<a class="stream-link" href="https://open.spotify.com/artist/NEW" ...>
```

Part 7 — (Bonus) Change the site COLOURS

All colours live in one spot. Open css/style.css; at the very top find the block starting with :root. Change a value, save, commit + push — it updates everywhere at once.

```
:root {
  --bg:      #060607; /* page background */
  --text:    #f2f0ea; /* main text */
  --accent:  #7d8b97; /* highlights & links - change me */
}
```

Note: the stylesheet is linked as style.css?v=2. If you change the CSS and an old version seems “stuck” in a browser, bump that number (?v=3) so visitors are forced to load the fresh file.

Part 8 — “Oops” — how to undo

While editing (not committed yet): press Ctrl+Z in VS Code. Or in GitHub Desktop, right-click the file in Changes → Discard changes to reset it completely.

Already committed & pushed? In GitHub Desktop open the History tab, right-click the bad commit → Revert changes. It creates a new commit undoing it; push, and the site returns to how it was.

Still stuck? Ask Claude — describe what you changed and want back.

Cheat sheet — what do I want to change?

I want to...	Open this file	Search for
Change homepage tagline / bio	index.html	EDIT: TAGLINE / EDIT: BIO
Change the homepage “Watch” video	index.html	the ID on the Watch iframe
Add a song's video	that door's file	the song title, then VIDEO_ID
Write a song's story	that door's file	the song title, then track-story

Change a song title	that door's file	EDIT: SONG TITLE
Add / replace a cover image	drop file into assets/images	(name it correctly)
Change a streaming link	index.html	EDIT: STREAMING LINKS
Change a social link	any page (footer)	EDIT: SOCIAL LINKS
Change site colours	css/style.css	:root

Remember the rhythm: Pull → edit & save → check the diff → Commit → Push → wait a minute → refresh gudulopes.com. That's every edit, forever.

Companion files in your site folder: [HANDOFF.md](#) (full technical reference) and [START-HERE_context.md](#). Site: gudulopes.com · Repo: [GLS9000/gudulopes-site](https://github.com/GLS9000/gudulopes-site).